Tiger Graphic Tees, Final Report

MGIS 445-01 Spring 2026
Joshua VanNostrand, Joseph Lombardo, Tim Salamone, Owen Cavo, Gleb Romanov

For our final project we built Tiger Graphic Tees, an online store for pop-culture graphic tees. The catalog has twelve shirts spread across six categories: Movies, Music, Anime, Gaming, TV, and Sports - and a shopper can browse the homepage, filter by category, open a product page, pick a size and color, add a shirt to their cart, and check out. The site also includes a contact form, a newsletter signup, and a small admin page that we use to demo the database in real time.

The site is built on PHP 8 and MySQL, served from RIT's `people.rit.edu` host with the database living on `studb-2.scb.rit.edu`. We chose to write everything in plain HTML and CSS without a framework or build step, so the project stays close to what the class covered and is easy to inspect by reading the source. Every page on the site is a standalone PHP script that opens a connection to the database, runs a query for whatever it needs, and prints the result as HTML. The same script handles different cases through query-string parameters: `shop.php?cat=anime` shows anime shirts, `product.php?id=4` opens the fourth product, and so on.

The data is organized into ten tables. The core relationship runs from `categories` to `products` and out to two child tables, `product_sizes` and `product_colors`, which let us store the available sizes and colors per shirt without repeating data. The shopping flow uses a `cart_items` table keyed by the visitor's session ID, so each browser keeps its own cart without needing an account. When a customer checks out, we write a row to `orders` along with one row per cart line in `order_items`, and we wrap those writes inside a single SQL transaction so that an order is either fully saved or not saved at all. The remaining tables: `users`, `contact_messages`, and `newsletter_subscribers` - stand on their own and capture the rest of the customer-facing inputs.

Every form on the site posts to a small PHP file in our `api/` folder. That file validates the input, checking that emails are well-formed, that the cart quantity is between one and ten, that the topic on the contact form is one of our allowed options, and then runs a single prepared statement to insert or update the right table. We use prepared statements throughout so user input is never concatenated into SQL, and we run every value we display through `htmlspecialchars` so that nothing posted into a contact message can inject markup or scripts back into the page. Cart reads and writes always include a `WHERE session_id = ?` clause, which keeps each visitor's cart isolated from everyone else's.

The site is live at **https://people.rit.edu/gar3381/445shirts**, and the admin demo page sits at `/admin.php` with the password `tiger445`. From there you can see live row counts for every table, the entity-relationship diagram of the schema, every column in every table pulled directly from MySQL, and a browser for the actual rows which is useful for showing that a contact-form

submission really does land in `contact_messages`, or that a placed order really does land in `orders` plus `order_items`. We worked out most of the design over Zoom and after-class meetings, splitting the page templates, schema, admin console, and deploy script across the team and stitching the pieces together so that the same patterns; prepared queries, escaped output, session-aware cart, repeat consistently across the whole codebase.